

TheBotCompany: Self-Organizing Multi-agent Systems for Continuous Software Development

Wenhan Lyu

William & Mary
Williamsburg, USA
wlyu@wm.edu

Yixuan Zhang

William & Mary
Williamsburg, USA
yzhang104@wm.edu

Yue Xiao

William & Mary
Williamsburg, USA
yxiao05@wm.edu

Yifan Sun

William & Mary
Williamsburg, USA
ysun25@wm.edu

Abstract

Large language model (LLM)-based multi-agent systems have shown promise in automating software development tasks. However, most vibe-coding systems focus on completing small tasks and incremental code changes, leaving persistent, continuous software development largely unexplored. We present THEBOTCOMPANY, an open-source orchestration framework for continuous multi-agent software development. THEBOTCOMPANY introduces three key innovations: (1) a three-phase state machine (**Strategy** → **Execution** → **Verification**) for milestone-driven development, (2) self-organizing agent teams where manager agents dynamically hire, assign, and retire worker agents based on project needs, and (3) asynchronous human oversight. We evaluate THEBOTCOMPANY on real-world software projects over multiple days of continuous development, measuring team adaptation patterns, milestone completion rates, cost efficiency, and code quality. Our results demonstrate that the self-organizing approach enables effective long-term software development with measurable progress, while the verification phase catches defects that would otherwise persist.

CCS Concepts

• **Software and its engineering** → **Software creation and management**.

Keywords

LLM-based software engineering, Continuous software development, Multi-agent systems

ACM Reference Format:

Wenhan Lyu, Yue Xiao, Yixuan Zhang, and Yifan Sun. 2026. TheBotCompany: Self-Organizing Multi-agent Systems for Continuous Software Development. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3832783.3834358>



This work is licensed under a Creative Commons Attribution 4.0 International License. ASE '26, Munich, Germany

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
<https://doi.org/10.1145/3832783.3834358>

1 Introduction

Software development is fundamentally a continuous process [19, 23, 31]. Real-world projects evolve over weeks, months, and years through iterative cycles of planning, implementation, testing, and revision [8, 16, 30]. Engineers join and leave teams, priorities shift as requirements change, and the scope of work adapts in response to emerging constraints and discoveries [21, 40, 57]. Recent Large language model (LLM)-based tools have made rapid progress on code generation and repair [9, 53], and multi-agent frameworks such as ChatDev [46] and MetaGPT [24] have shown that organizing LLM agents into role-specialized teams can yield functional software from natural language specifications. However, these systems mostly operate in isolated, session-scoped interactions where a user provides a prompt, the system generates code in minutes, and the session ends with limited ability to maintain context across sessions, track cumulative project-level progress, or resume and iterate on prior work [6, 17, 26].

We identify three specific gaps in existing approaches.

- **First, limited persistent orchestration:** current systems (e.g., ChatDev [46], MetaGPT [24], and single-agent tools such as SWE-Agent [53] and OpenHands [52]) are primarily designed for single-session execution (each run addresses one task and terminates). While agents can be re-invoked on the same repository, existing frameworks lack built-in mechanisms to track cumulative progress toward milestones, carry forward verification feedback, or automatically resume work after interruptions [11, 22, 42]. The missing coordination state makes long-horizon projects that require iterative refinement over days or weeks difficult to sustain, as the coordination context must be manually reconstructed between sessions.
- **Second, static or session-scoped team composition:** In ChatDev, MetaGPT, AgileCoder [39], and ALMAS [49], agent roles are predetermined by the framework designer and remain fixed throughout execution. Recent work on dynamic composition—AgentVerse [14], DyLAN [35], and Self-Organized Agents [27]—adjusts team membership within a single session, but does not persist team adaptations across development cycles or adapt workforce composition in response to verification feedback from prior milestones. Yet real software teams routinely evolve their composition as projects progress—adding a security specialist when a vulnerability surfaces, or retiring a prototyping role once

architecture stabilizes [21]. Bridging this gap requires a team composition that is both dynamic *and* persistent across the lifecycle of a project.

- Third, **synchronous human oversight**: Existing systems support human involvement primarily through synchronous interaction: users either specify requirements before execution, monitor and intervene in real time during a session, or review outputs afterward [52]. However, real-world software oversight is largely asynchronous, meaning that clients file issues, and managers adjust priorities in a tracker, and developers consume this guidance at natural transition points without blocking ongoing work [1, 22]. Current frameworks lack a channel for humans to steer multi-day development asynchronously and allows agents to consume guidance at appropriate phase boundaries.

To address these gaps, we present THEBOTCOMPANY¹, an open-source orchestration framework for continuous, multi-agent software development. To enable *persistent orchestration*, THEBOTCOMPANY organizes development around *milestones*, discrete units of work that progress through a repeating three-phase lifecycle: a **Strategy** manager defines the next milestone with concrete objectives, an **Execution** manager assembles and adapts a team of worker agents to implement it, and a **Verification** manager independently evaluates the result, rejecting incomplete work with structured feedback to trigger corrective cycles. The three-phase design mirrors the plan–implement–review cycle established in iterative software engineering practice [54] and separates concerns so that no single agent both produces and judges its own output. This lifecycle repeats continuously, with full project state persisted across cycles, driving sustained progress across days of unattended operation.

To replace *static team composition*, agent teams in THEBOTCOMPANY are self-organizing: only the three manager roles are permanent, while managers dynamically hire, specialize, and retire worker agents based on milestone demands. Workers’ skills are defined by persistent skill files that managers generate or modify as project needs evolve, enabling the workforce to adapt without framework-level reconfiguration.

To support *asynchronous human oversight*, THEBOTCOMPANY provides a web-based monitoring platform through which human users can inspect project progress, review agent reports and communications, and file issues that manager agents consume at natural phase boundaries, which steer development without interrupting in-progress work. Human users can file issues that manager agents consume at subsequent phase boundaries, mimicking the natural workflow of the real-world software development process.

We evaluate THEBOTCOMPANY on four real-world software projects over days of continuous operation, along with benchmark-based evaluations to catch team adaptation, verification-driven quality improvement, failure recovery, and practical lessons for future multi-agent systems.

The contributions of this paper are:

- (1) We design and implement THEBOTCOMPANY, an open-source orchestration framework for persistent, milestone-driven multi-agent software development with asynchronous oversight.

- (2) We introduce a self-organizing team model in which managers hire, specialize, reassign, and retire workers as milestone demands evolve.
- (3) We evaluate THEBOTCOMPANY on four long-running projects and five hard ProjDevBench problems, and we derive design lessons for continuous multi-agent software development.

2 Related Work

LLM-based code generation. LLMs have demonstrated strong capabilities in code generation, progressing from function-level synthesis [5, 13, 33] to repository-level tasks [29, 53, 56]. Early work (e.g., Codex [13], AlphaCode [33]) demonstrated that AI could solve competitive programming problems and generate correct implementations from natural-language descriptions. Generative AI’s coding capabilities are being rapidly commercialized through tools such as GitHub Copilot [18], Claude Code [3], and OpenAI Codex [41]. Subsequent models, including StarCoder [32], CodeLlama [47], and DeepSeek-Coder [20], improved generation quality through larger training corpora and specialized fine-tuning. These advances provide the per-invocation generation capability that THEBOTCOMPANY’s agents rely on, but they do not address coordination, persistence, or quality assurance across extended development efforts, which motivate a THEBOTCOMPANY’s orchestration layer.

Single-agent software development systems. A growing body of work focuses on single-agent systems that autonomously perform software engineering tasks. SWE-Agent [53] introduced an agent-computer interface that enables an LLM to navigate repositories, edit files, and run tests to resolve GitHub issues, driving rapid progress on SWE-bench Verified [29]. Later work (e.g., OpenHands [52], RepairAgent [9]) generalized this model with broader agent architectures, sandboxed execution, and iterative program repair. However, all these systems operate within a single invocation session per task. Each run starts from a clean state, addresses one issue, and terminates, with no mechanism to maintain context across tasks, learn from prior resolutions, or coordinate multiple tasks toward a coherent development objective. Quality assurance remains limited to executing pre-existing test suites associated with the target issue, and no system implements an independent verification stage that evaluates whether produced changes satisfy broader project-level goals—a gap that THEBOTCOMPANY’s verification phase directly addresses.

Multi-agent software development systems. Multi-agent frameworks address the coordination gap by organizing LLM-powered agents into collaborative teams that mirror human software organizations. ChatDev [46] introduced a simulated software company with role-specialized agents (CEO, CTO, programmer, tester) communicating through structured dialogue along a waterfall pipeline. Later frameworks refined this model in different ways. For example, MetaGPT [24] replaced freeform chat with shared artifacts governed by Standard Operating Procedures, AgileCoder [39] introduced sprint-based iterative refinement, and ALMAS [49] aligned agents with agile roles to cover end-to-end software lifecycle stages. In all these systems, however, team composition is fixed at design time: roles are predetermined by the framework designer and remain static throughout execution, preventing the system from adapting its workforce as project demands evolve.

¹<https://github.com/syifan/thebotcompany>

Dynamic agent team composition. A growing body of work explores dynamic agent composition beyond fixed-role teams. AgentVerse [14] and AutoAgents [12] adjust group membership or generate custom teams during task-solving based on progress or task descriptions. DyLAN [35] refines this with a principled agent selection algorithm based on unsupervised importance scores. Self-Organized Agents [27] introduces hierarchical spawning, where “mother” agents create specialized “child” agents scaled to problem complexity. MegaAgent [51] and Think-on-Process [34] go further, generating agent roles and development processes entirely from project requirements without predefined SOPs. ChatDev’s Experiential Co-Learning [44] and Iterative Experience Refinement [45] take a complementary approach, transferring task-solving experience across a series of projects. These systems represent the closest precursors to THEBOTCOMPANY’s self-organizing approach. However, all operate within a single execution session or batch of independent tasks: none maintain team state across development cycles, adapt workforce composition in response to verification feedback from prior milestones, or implement an independent verification stage that can reject deliverables and trigger corrective iterations.

To the best of our knowledge, THEBOTCOMPANY is the first framework to combine all three: (1) team persistence *across* milestones rather than within a single session, (2) workforce adaptation driven by verification feedback from prior milestones, and (3) an independent adversarial verification phase that can reject deliverables and trigger corrective cycles. Prior dynamic-composition systems provide at most one of these in isolation, which is the gap our design and evaluation target.

3 THEBOTCOMPANY

3.1 Architecture Overview

THEBOTCOMPANY organizes continuous development around these concepts (see Figure 1 for an overview):

A project corresponds to a software repository under active development. Each project maintains persistent memory across sessions through a layered storage model (orchestration state, milestone history, an issue tracker with inter-agent comments, cycle reports, and a cost ledger), allowing it to survive restarts, resume mid-milestone, and provide agents with a complete, auditable record of prior decisions. Multiple projects run simultaneously on a single THEBOTCOMPANY instance, each progressing independently with fully isolated memory.

A milestone is the fundamental unit of work in THEBOTCOMPANY, representing a discrete development objective. As real-world software projects are too large for any single agent invocation to address end-to-end, milestones break a long-term project goal into manageable pieces that can each be planned, implemented, and verified independently. A dedicated agent team (**Athena**) is responsible for milestone planning. Milestones are organized hierarchically using dotted numbering (e.g., 1, 1.1, 1.1.2). The hierarchical design allows the system to automatically adapt milestone granularity to the agents’ capabilities by breaking unachievable milestones into smaller tasks.

The orchestrator is the central coordination layer, implemented in JavaScript and running on Node.js. It drives each project through

development cycles (see §3.2), invoking agents according to the current phase, tracking progress, and managing phase transitions. The orchestrator does not make software engineering decisions itself; rather, it provides the scheduling infrastructure that enables agents to operate autonomously within a structured lifecycle. It is responsible for enforcing time limits, detecting failures, and maintaining persistent state across cycles.

The harness is responsible for directly interacting with LLMs. We use the `pi-ai` library [55] for unified multi-provider LLM access, streaming, tool calling, and per-call token tracking. On top of `pi-ai`, THEBOTCOMPANY adds automatic context compaction, tiered model routing, retry logic with exponential backoff, and structured directive parsing from agent responses.

Agents are the autonomous actors that perform all software engineering work. Each agent is an LLM-based coding agent with access to a project’s repository, command-line tools, and a structured communication channel backed by an internal issue tracker. Agents are divided into two categories:

Manager agents are three permanent agents, each assigned to a distinct phase of the milestone lifecycle: **Athena** (planning), **Ares** (implementation), and **Apollo** (verification)². Each milestone passes through all three managers in sequence. Managers make strategic and organizational decisions by defining milestones, assembling teams, assigning tasks, and evaluating outcomes.

Worker agents are the autonomous actors that perform all assigned software engineering work. Each worker is an LLM-based coding agent, and managers may assign different models to workers. A worker is visible only to their manager, enforcing phase-scoped team isolation. Workers perform the actual software engineering tasks: writing and refactoring code, running test suites, opening pull requests, and producing artifacts. The orchestrator executes scheduled workers serially within each cycle, as the goal of THEBOTCOMPANY is long-horizon, limited-human-involvement software development, where development time is not critical.

Team management is achieved by managers creating, assigning, and retiring workers in response to milestone needs. Each worker is defined by a *skill file*. A skill file is a Markdown document with YAML front matter that specifies the worker’s role (e.g., “backend developer,” “test engineer,” “documentation writer”), its reporting manager, LLM model tier, and tool permissions. The Markdown body records responsibilities, conventions, and known pitfalls and is updated by the owning agent across cycles. Skill files persist in the project workspace across cycles.

Budget management enforces a user-configured 24-hour rolling spending limit, as autonomous agents invoking commercial LLM APIs can incur substantial costs if left uncontrolled. When a project falls below a safety threshold, the orchestrator introduces adaptive sleep intervals between agents to spread expenditure over the remaining time horizon; once the budget is fully exhausted, it sleeps until the budget replenishes at the start of the next window before resuming automatically.

Agent communication is achieved within THEBOTCOMPANY through an issue system (similar to GitHub issues and comments).

²We use names from Greek mythology for the three manager roles throughout this paper for clarity of exposition.

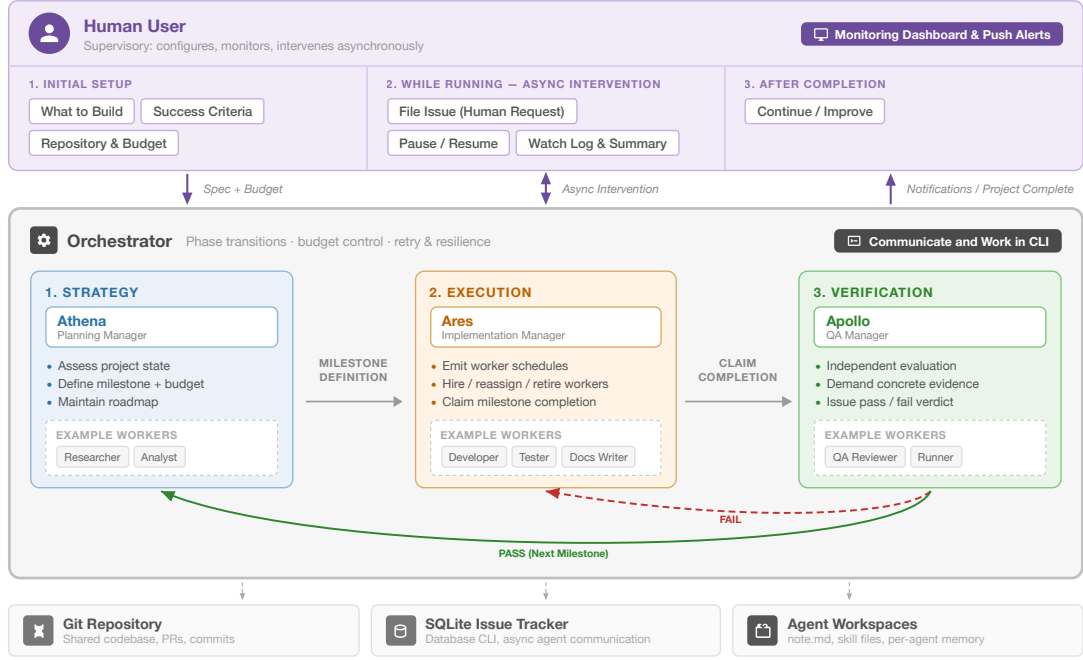


Figure 1: Architecture of THEBOTCOMPANY. The human monitors the system through a dashboard connected to the orchestrator, which drives a three-phase milestone lifecycle (**Strategy** → **Execution** → **Verification**). Manager agents govern each phase and dynamically hire/retire worker agents. Solid arrows show normal progression; dashed arrows show recovery paths. All agents access shared infrastructure through a common bus.

Algorithm 1 Orchestrator main loop.

```

Require: Project  $P$ 
1: loop
2:    $\text{milestone, budget} \leftarrow \text{Athena's Team.Plan}(P)$ 
3:   repeat
4:      $\text{Ares's Team.Execute}(P, \text{milestone, budget, cycle})$ 
5:      $\text{verdict} \leftarrow \text{Apollo's Team.Verify}(P, \text{milestone})$ 
6:     if  $\text{verdict} \neq \text{PASS}$  then
7:        $\text{budget} \leftarrow \lfloor \text{budget}/2 \rfloor$ 
8:     end if
9:   until  $\text{verdict} = \text{PASS}$  or  $\text{budget} = 0$ 
10: end loop

```

The **human user** configures a project by answering two specification questions: (1) *What do you want to build?* and (2) *How will you determine whether the project is successful?*, along with a repository path and budget. Beyond this setup, users can monitor, supervise, and raise issues in THEBOTCOMPANY to provide guidance.

3.2 Three-Phase Milestone Lifecycle

Software development in THEBOTCOMPANY is organized around *milestones*, discrete units of work with defined objectives and cycle budgets. Projects progress through a repeating three-phase lifecycle: **Strategy**, **Execution**, and **Verification** (see Algorithm 1 for the main loop of THEBOTCOMPANY), modeled on the plan – implement – review cycle of real-world software engineering practice for improved multi-agent performance [54].

The **Strategy phase**. **Athena** keeps the project moving toward the user-defined goal. Drawing on the project specification, the

issue tracker (which may contain user intervention requests), the codebase, and any feedback from prior verification failures, **Athena** assesses overall progress, reconciles new requirements with the existing plan, defines a concrete next milestone with a title, description, and cycle budget, and updates a persistent roadmap.

The **Execution phase**. **Ares** receives the milestone definition and is responsible for achieving it within the allocated cycle budget. Each cycle, **Ares** assesses progress and emits a *schedule*, which is an ordered list of workers and their tasks. When **Ares** judges the milestone complete, it issues a *completion claim* and the orchestrator transitions the project to the **Verification** phase. If the budget expires before completion, control returns to the **Strategy** phase. Algorithm 2 details how **Ares's** team executes a milestone. **Athena's** and **Apollo's** teams follow a similar cycle structure but without a cycle budget, as they complete in a single cycle.

The **Verification phase**. **Apollo** independently evaluates whether the claimed milestone has been achieved. It may assemble a verification team to run tests, inspect code, or perform quality checks, then issue a **pass** or **fail** verdict with structured feedback. On a pass, the system advances to the next **Strategy** phase. On a fail, if remaining budget permits, the system returns to **Execution** with the budget halved and **Apollo's** feedback, giving the execution team a bounded opportunity to address deficiencies. The budget halves on each subsequent failure, progressively tightening the fix window; once exhausted, the milestone is marked failed, and control returns to **Athena**. Halving bounds the correction attempts for a milestone with initial budget b to at most $\lfloor \log_2 b \rfloor$ rounds, giving

Algorithm 2 *Ares*’s team execution cycles. Other teams follow a similar structure without a cycle budget.

```

Require: Project  $P$ , milestone  $m$ , budget  $b$ 
1: for  $cycle = 1$  to  $b$  do
2:    $schedule \leftarrow Ares.Assess(P, m, b, cycle)$ 
3:   if Ares claims completion then
4:     break
5:   end if
6:   for each item  $i$  in  $schedule$  do
7:     if  $i$  is a delay then
8:        $sleep(i.duration)$ 
9:     else
10:       $i.worker.Execute(P, i.task)$ 
11:    end if
12:  end for
13: end for

```

a deterministic upper bound on per-milestone cost and preventing indefinite cycling on contested verifications. The rule follows the exponential-backoff convention of retry scheduling [38] rather than an optimality argument; we adopt it as a heuristic with a bounded-cost guarantee, not a claim of convergence to an optimal fix schedule. *Apollo* is designed to be *adversarial* to demand concrete evidence, such as test output or CI results, rather than accepting claims at face value, mirroring the separation between development and QA in human software organizations.

Structured directives. Manager agents communicate decisions to the orchestrator through structured directive blocks embedded in their natural language responses. These directives follow a defined schema and enable machine-parseable coordination:

- *Milestone directives* (*Athena*) specify the milestone title, description, and cycle budget.
- *Schedule directives* (*Ares* and *Apollo*) specify workers, their tasks, optional per-agent delays, and visibility modes.
- *Completion claims* (*Ares*) signal that the milestone objectives have been met.
- *Verification verdicts* (*Apollo*) record pass/fail decisions with structured feedback identifying specific deficiencies.
- *Project completion directives* (*Athena*) signal that the project’s ultimate goal has been fully achieved, causing the orchestrator to pause and notify the user.

The orchestrator parses these directives to drive phase transitions and schedule worker invocations. Manager agents do not communicate with each other directly. Instead, all cross-phase information is mediated by the orchestrator. Specifically, verification feedback from *Apollo* is injected by the orchestrator into *Ares*’s prompt context at the start of each fix round, and each manager receives the current milestone definition and relevant orchestration state in its context when its phase begins. This design keeps coordination logic within the agents’ natural language output while providing the structure needed for automated orchestration.

3.3 Agent Teaming

Self-organizing agent teams. Only the three manager roles are permanent, while the worker team is entirely at each manager’s discretion. Workers are defined by *skill files*. Each manager may only hire and schedule workers who report to them: *Athena* may assemble a temporary research team, *Ares* composes developers

and testers, and *Apollo* staffs QA specialists. We find that the semi-structured design best combines the human wisdom (predefined structure) and AI intelligence (guides concrete worker jobs). The design can maintain a reasonable workflow while keeping the team flexible to the task at hand.

Information control. Managers control not only *what* workers do but also *what they see*. When scheduling a worker, a manager specifies a *visibility mode*: **Full** exposes all open issues; **Focused** exposes only a manager-specified subset (reducing irrelevant context); and **Blind** provides no issue board context, useful for independent verification tasks where prior discussion might bias the worker. The orchestrator propagates the chosen mode via an environment variable injected into the agent’s invocation, which the `tbcd` CLI respects when responding to issue queries.

Coordination. Agent coordination is backed by a per-project SQLite database. SQLite is preferred over cloud-hosted trackers (e.g., GitHub Issues) for three reasons: (1) local queries complete in microseconds versus hundreds of milliseconds for remote API calls; (2) cloud rate limits become binding when multiple agents are active across several projects simultaneously; and (3) agent’s internal deliberation (e.g., failed attempts, experimental branches) should not pollute the project’s public issue tracker or inadvertently trigger notifications by @-mentioning real users who share agent names.

Issue tracking. Agents communicate asynchronously through an issue tracker modeled on GitHub Issues. The schema stores issues (with title, description, status, and assigned agent), threaded comments, milestone history, complete per-invocation agent reports (including parsed directive blocks), and the active agent registry. Managers open issues to define work items; workers post comments with progress updates; the verification team records findings. Agents access the database through `tbcd`, a CLI injected into every agent environment whose commands follow natural conventions (e.g., `tbcd issue list --status open`) and produce human-readable output that fits within an agent’s context window.

Agent-local knowledge base. Because each agent invocation starts with a fresh context window, agents face a cold-start problem. To mitigate this, the orchestrator injects a persistent `note.md` into each agent’s context on every invocation. Agents update this file freely to record conventions, known pitfalls, and architectural decisions, and may create additional workspace files (checklists, analysis reports, acceptance criteria logs) that accumulate across cycles into a role-tailored knowledge base.

3.4 Monitoring Dashboard

THEBOTCOMPANY includes a Progressive Web App (PWA) monitoring dashboard served by the orchestrator’s built-in HTTP server (no separate backend required), communicating via REST and a Server-Sent Events (SSE) stream for real-time updates. Web Push (VAPID) notifications alert users to milestone completions, verification failures, or system errors on mobile devices even when the browser is closed, supporting unattended operation.

Process inspection. The dashboard displays current phase, active milestone, cycle counter and budget remaining, per-agent status, and cost metrics. A key capability is *live agent streaming*: while an agent runs, its console output, including reasoning, tool calls,

file edits, and command execution, is streamed in near real time, letting users understand *how* decisions are reached rather than just their outcomes. Completed reports are archived and filterable per agent; an open-PRs panel aggregates pull requests created by agents across all projects.

Intervention and emergency reset. Users steer development by filing issues in the project tracker, the same mechanism agents use, which manager agents pick up asynchronously in subsequent cycles. Strategic redirection flows through the management hierarchy (*Athena* consumes user issues first) rather than disrupting in-progress workers. When the system enters a degenerate state (e.g., repeated verification failures cycling without progress), the *bootstrap* operation immediately halts all agents, clears the current milestone, and resets the phase to *Strategy* so the user can file corrective guidance before *Athena* re-plans from scratch.

4 Evaluation

We evaluate THEBOTCOMPANY through four long-running software deployments over multiple days of continuous, largely unattended operation. We treat the four long-running deployments as a *feasibility and behavioral-characterization* study. These studies demonstrate that self-organizing continuous development is achievable, and we use them to characterize how THEBOTCOMPANY behaves. The studies cannot claim statistical generality, and we complement them with a controlled benchmark comparison that is externally scored.

4.1 Research Questions

RQ1: Can a self-organizing multi-agent system make progress toward defined quality objectives over multi-day continuous operation? We measure whether THEBOTCOMPANY can make cumulative progress (completing milestones, resolving issues, and growing codebases) without reverting to a degenerate state where cycles produce no useful work.

RQ2: How does the self-organizing multi-agent approach compare against single-agent baselines on hard end-to-end development tasks? A core motivation of THEBOTCOMPANY's design is that coordinated multi-agent execution should outperform single-agent approaches on development tasks requiring prolonged effort and iterative refinement. We evaluate THEBOTCOMPANY against Claude Code on the five hardest problems from ProjDevBench [36], a recent end-to-end benchmark that evaluates coding agents on system architecture design and functional correctness in repositories with minimal external intervention. The comparison isolates the contribution of multi-agent coordination from model capability by holding the underlying LLM constant in one condition while varying models in the single-agent baseline.

RQ3: How does the verification phase behave in practice, and what defects does it surface? The three-phase lifecycle introduces an independent verification step that may reject an ostensibly completed milestone and trigger a fix round. We conduct a systematic log-based analysis of verification behavior across all observed milestones, providing an empirical characterization of whether the verification step catches substantive defects rather than being a passive pass-through.

RQ4: What is the cost profile of continuous multi-agent development, and how does token efficiency compare to single-agent baselines? Autonomous development using commercial LLM APIs incurs a direct monetary cost that scales with system complexity. We analyze the cost structure of THEBOTCOMPANY across the projects run in our experiments: cost per milestone, token consumption breakdown by agent role (manager overhead versus worker productivity), and the effectiveness of the budget-aware scheduling mechanism in preventing overruns. For comparative purposes, we report total token consumption for ProjDevBench problems across THEBOTCOMPANY and Claude Code, providing a quantitative measure of cost-efficiency relative to single-agent execution on identical tasks.

RQ5: What are the dominant failure modes, and how effective are the recovery mechanisms? Long-running systems experience diverse failures, including agent timeouts, model errors, incorrect code, and infrastructure disruptions. We categorize observed failures, measure their frequency, and evaluate how effectively the recovery mechanisms (timeout escalation, consecutive-failure detection, cycle accounting, commit-early policy) preserve progress and prevent resource waste.

4.2 Experimental Setup

4.2.1 Subject Projects. We evaluate THEBOTCOMPANY on two categories of experiments: a standardized benchmark and long-running real-world projects.

Benchmark evaluation. We select the five problems designated as the hardest in ProjDevBench [36], an end-to-end coding benchmark that evaluates agents on full project development from requirements, assessing system architecture design, functional correctness, and iterative refinement. Each problem targets a C++ implementation and is evaluated on two dimensions: an execution score (automated Online Judge (OJ) testing, 80 points maximum) and an overall score (100 total) that adds LLM-assisted code review (20 points). We run THEBOTCOMPANY under identical conditions to those prescribed by ProjDevBench, where coding agents can retry when an OJ test fails, up to a predefined maximum number of attempts.

Long-running projects. We choose four projects of varying character, complexity, and implementation language.

- **Project A (M2Sim):** A cycle-accurate, execution-driven, Apple M2 CPU simulator written in Go, targeting correct execution of all benchmarks in the PloyBench [43] and PARSEC [7] suites with timing estimation error below 20% on average and 50% per benchmark.
- **Project B (GroundDB):** A from-scratch, Python relational database engine with no external database or SQL parsing dependencies, targeting TPC-H benchmark [15] correctness.
- **Project C (RustLaTeX):** A $\text{\LaTeX}$ compiler in Rust, targeting document parsing, layout computation, and PDF output similarity with other $\text{\LaTeX}$ compiler outputs.
- **Project D (PyInterpreter):** A Python interpreter implemented in C++. Note that Project D is drawn from the ProjDevBench problem set, providing a direct bridge between the benchmark evaluation and the long-running project track.

These projects span four implementation languages (Go, Python, Rust, C++) and four domains (systems simulation, database engines, tooling, compiler/interpreter), covering a range of project durations and complexity levels.

Each project is configured with a brief (20–80 words) natural-language specification answering only *what to build* and *how success is defined*, mirroring realistic usage in which users give short instructions and refine them later through issues. The four specifications fall into three success-criterion regimes: **executable oracles** authored by the user (Project A: PloyBench [43]/PARSEC [7] timing bounds; Project B: TPC-H queries), an **online-judge oracle** reused from ProjDevBench (Project D), and a **qualitative target** (Project C: “*binary-identical PDF*”). As an example, the complete M2Sim (Project A) specification is:

What to build. M2Sim is a cycle-accurate, execution-driven Apple M2 CPU simulator built on the Akita framework [28].
Success criterion. Support simulating the whole PloyBench benchmark suite and the PARSEC benchmark suite with 20% average time-estimation error; the maximum error of any single benchmark must not exceed 50%, where $error = |sim - hw| / \min(sim, hw)$.

4.2.2 Baselines. For the ProjDevBench evaluation, we compare THEBOTCOMPANY against Claude Code instantiated with five model backends evaluated in the original benchmark, spanning both closed- and open-weight frontier models. THEBOTCOMPANY is restricted to Claude Sonnet 4.5 as its backend, prohibiting other models to ensure a fair comparison. All agents invoke their models with provider-default decoding parameters (temperature, top- p , etc.); as default decoding is both the fair-comparison condition against Claude Code under the same base model and the deployment-realistic baseline a practitioner would adopt without prior tuning data. ProjDevBench is chosen as the primary evaluation vehicle because it shares the same operational premise as THEBOTCOMPANY: given a repository initialized from scratch, the system must develop it autonomously with only limited feedback from an online judge, whereas the majority of existing code agent benchmarks (e.g., SWE-bench [29]) focus on resolving discrete, pre-existing issues in established codebases, a fundamentally different task structure.

For the long-running project evaluation, no equivalent single-agent baseline exists, as running multi-day development sessions with Claude Code on the same proprietary projects under controlled conditions was not feasible; cost and workflow analysis for these projects is therefore self-contained rather than comparative.

4.2.3 Intervention Protocol. To test whether THEBOTCOMPANY sustains continuous operation without human guidance, and to keep runs reproducible, we supplied *no* human instructions during the four long-running projects and the benchmark runs beyond the initial two-question specification. The sole exception is ProjDevBench’s OJ feedback, which the benchmark protocol permits. We route this feedback through the same asynchronous human-in-the-loop channel a real user would use: after each completion claim, we write the OJ response into the project issue tracker as a filed issue and resume the run, so *Athena* consumes it at the next **Strategy** phase exactly as it would a user-filed issue. This exercises the async oversight channel end-to-end (analyzed in §4.3.2) while keeping the long-running projects fully unattended.

4.2.4 Metrics.

- **Progress metrics:** Milestones completed, commits, lines of code added or modified, test cases added, and benchmark scores (execution score and overall score for ProjDevBench problems).
- **Verification behavior:** *Apollo* invocation count, verification rejection rate, defect categories identified at rejection, fix-round iteration count, and milestone-level rework patterns extracted from orchestrator logs.
- **Cost and token metrics:** Total token consumption, cost per milestone, token breakdown by agent role (manager versus worker), and budget scheduling effectiveness.
- **Reliability metrics:** Timeout rate, consecutive failure events, cycle waste ratio (failed cycles divided by total cycles), and mean cycles to recovery per failure episode.

All metrics are collected automatically from the SQLite communication stores, cost tracking logs, Git history, and server logs maintained by THEBOTCOMPANY during operation.

4.3 Results

All four long-running projects and all five ProjDevBench runs reached an agent-claimed success state within their cycle budgets; the analyses below characterize *how* that state was reached, including the rework and recovery behavior along the way.

4.3.1 RQ1: Continuous Development Effectiveness. Table 1 summarizes the four long-running deployments. Across all projects, THEBOTCOMPANY completed 164 milestones and 616 agent cycles spanning 142 wall-clock hours, reaching the primary quality objective in every case without human intervention. No project exhibited the degenerate behavior we guarded against. Cycle waste ratios were low and only occurred in two projects: 5.3% in M2Sim, and in RustLaTeX, 27 fix-round milestones account for a 6.9% waste ratio. The projects also show that THEBOTCOMPANY adapts to widely varying task complexity without structural change: GroundDB completed in 17 cycles with a tight correctness oracle, while RustLaTeX required 392 cycles as each layout change interacted with competing rendering constraints. Both trajectories exhibit forward progress rather than oscillation.

Finding (RQ1): *TheBotCompany sustained forward progress across all four long-running projects, spanning four languages (Go, Python, Rust, C++) and three architectural paradigms, completing 164 milestones, 616 cycles, and 142 wall-clock hours without human intervention beyond initial project configuration.*

4.3.2 RQ2: Comparison with Single-Agent Baselines. Table 2 reports scores on the five hard problems from ProjDevBench. THEBOTCOMPANY (sole Claude Sonnet 4.5) achieves a mean execution score of 53.2 and overall score of 70.0, leading all five single-agent baselines. The gap over Claude Code with the *same* underlying model (Sonnet 4.5: 43.6 exec, 59.3 overall) is +9.6 and +10.7 points respectively, directly attributable to multi-agent coordination rather than model capability, since the LLM backend is held constant.

The advantage is most pronounced on the hardest problems. On P3 and P17, every single-agent system scores near zero (≤ 7.5), while THEBOTCOMPANY scores 48.0 and 22.4—problems where a one-shot session cannot traverse the implementation space that THEBOTCOMPANY’s milestone structure explores iteratively. On

Table 1: Long-Running Project Deployments.

Project	Language	Milestones	Cycles	Cost	Duration (hours)	Cycle/Milestone	\$/Milestone	hour/Milestone
A (M2Sim)	Go	12	105	\$527	49.1	8.8	43.9	4.09
B (GroundDB)	Python	4	17	\$33	2.6	4.3	8.25	0.65
C (RustLaTeX)	Rust	95	392	\$700	62.9	4.1	7.37	0.66
D (PyInterpreter)	C++	53	102	\$261	27.8	1.9	4.92	0.52

Table 2: ProjDevBench Results: Execution Score (max 80) and Overall Score (max 100). THEBOTCOMPANY (BotCo) uses Claude Sonnet 4.5 as its sole backend during these tasks. All other models run in Claude Code. Bold indicates column maximum.

#	BotCo	Sonnet 4.5	DeepSeek-v3.2-exp	GLM-4.6	GPT-5	Kimi-K2
<i>Execution Score (max 80)</i>						
P3	48.0	0.0	0.0	0.0	0.0	0.0
P4	61.4	68.2	9.0	7.2	43.8	0.0
P15	71.1	62.2	44.4	62.2	71.1	62.2
P16	62.9	80.0	80.0	80.0	80.0	80.0
P17	22.4	7.5	7.5	7.5	7.5	0.0
Avg.	53.2	43.6	28.2	31.4	40.5	28.4
<i>Overall Score (max 100)</i>						
P3	66.7	18.0	18.0	18.0	18.7	18.7
P4	69.8	78.6	18.6	18.4	54.6	0.0
P15	89.8	80.9	62.4	80.9	90.4	80.9
P16	82.2	98.0	98.7	98.0	97.3	97.3
P17	41.4	21.0	26.0	26.0	24.5	19.0
Avg.	70.0	59.3	44.7	48.3	57.1	43.2

P16, the result reverses: THEBOTCOMPANY scores 62.9 execution versus 80.0 for all Claude Code baselines. Post-hoc analysis of P16 logs reveals that its specification admits a compact, self-contained solution, while THEBOTCOMPANY’s multi-milestone decomposition introduced architectural layering that added verification overhead without proportional benefit, suggesting that multi-agent coordination has diminishing returns when the one-shot solution space is already effective.

Finding (RQ2): THEBOTCOMPANY (Sonnet 4.5) leads all single-agent baselines on ProjDevBench, gaining +9.6/+10.7 points over the same-model baseline (Sonnet 4.5) with Claude Code on execution/overall score. The advantage concentrates on high-difficulty problems where iterative multi-milestone refinement is essential.

Because ProjDevBench permits bounded retries with OJ feedback, RQ2 also exercises THEBOTCOMPANY’s asynchronous oversight channel: each OJ verdict is filed as a project issue and consumed at the next Strategy phase (§4.2.3). Across all five problems, Athena responded by triggering fresh planning and implementation rather than treating the verdict as a passive signal; the compact P15 redesign (§4.3.5) is one outcome of this loop.

4.3.3 RQ3: Verification Phase Behavior. Table 3 reveals that Apollo’s rejection rate varies systematically with task specification clarity, not with project scale. Projects with exact, automatically evaluable acceptance criteria (Project B and D) produced 0% rejection: Apollo acted as a confirmatory gate rather than a corrective one (i.e., it confirmed already-passing work rather than blocking and triggering rework cycles). Projects with iterative quality targets (Project C:

Table 3: Verification Phase Behavior Across Projects (RQ3). Reject rate = `verify_fail` / total Apollo verdicts.

Project	Apollo Invocation Count	Reject	Dominant Failure Mode
A (M2Sim)	19	5.3%	Accuracy regression
B (GroundDB)	4	0.0%	NA
C (RustLaTeX)	97	18.6%	Similarity regression
D (PyInterpreter)	27	0.0%	NA

pixel similarity compared with pdf`latex`’s [50] output PDF thresholds that shift per milestone; Project A: timing-accuracy bounds across 30 benchmarks) produced substantial rejections, with Apollo catching regressions that would otherwise have shipped (18.6% of verifications in RustLaTeX and 5.3% in M2Sim).

Project C further demonstrates that THEBOTCOMPANY can autonomously construct a grounded evaluation target when the user’s specification is ill-defined. The project specification stated “*generate binary identical PDF*”; Athena determined early that strict binary identity was unachievable given font-rendering and layout nondeterminism in the toolchain. Rather than stalling, Athena defined a tractable proxy at milestone M29: pixel similarity against pdf`latex`’s output on the same .tex files. The M29 issue reads: “*Establish the ability to compare our compiler’s output against pdf`latex` output in CI. This is critical for measuring progress toward the ‘binary identical PDF’ project goal.*” This self-defined metric then served as the acceptance criterion for all 97 subsequent verification cycles. The project terminated at 99.21% average pixel similarity after 95 milestones with an explicit self-aware stopping statement: “*All strategic improvement approaches have been exhausted... The remaining 2.12% gap... is dominated by the mega-line layout problem which cannot be fixed without regression—every approach tried (9+ confirmed anti-patterns) consistently regresses similarity.*”

In Project C, the 18.6% rejection rate is not evidence of wasted effort: Apollo’s rejections directly drove the accumulation of a growing list of documented *anti-patterns* in Athena’s planning context—approaches confirmed to cause pixel-similarity regressions. This list constrained Athena’s future search space across subsequent milestones. 27 dedicated fix-round milestones produced measurable recovery: whenever a regression milestone was immediately followed by a revert, the similarity score returned to or exceeded the pre-regression baseline.

Case Study (Project C, RustLaTeX). We trace a five-milestone episode from Project C that shows the verification-planning feedback loop driving both regression recovery and eventual termination. By M91, eight documented anti-patterns had accumulated in Athena’s planning context—layout interventions confirmed to regress pixel similarity—and the baseline stood at 97.88%.

At M92, *Athena* reasoned that applying the same layout technique (AlignmentMarker insertion) to *section headings* rather than paragraphs was mechanically distinct from the documented anti-pattern and proposed it as a fix. The milestone passed *Apollo*'s unit-test gate, but *Athena*'s next planning cycle (M93) detected a CI-level similarity regression to 97.51%. *Athena* immediately scheduled a revert milestone; *Apollo* confirmed recovery to the 97.88% baseline, and the anti-pattern list grew to nine entries. Seventeen cycles later, M95 reproposed the same class of fix with adjusted parameters (27.9 pt instead of 21 pt). This time *Apollo* explicitly failed the milestone:

"M95 FAILED: compare.tex regressed from 97.88% to 97.31% (−0.57%). AlignmentMarker for section headings is a confirmed dead end regardless of line-height value."

Rather than searching for a tenth variant, *Athena* declared the project complete, which is a self-aware stopping decision grounded in exhausted hypothesis space.

This episode illustrates three properties of the verification-planning loop: (1) *Apollo*'s unit-test gate can miss CI-level regressions, making *Athena*'s planning cycle a necessary second quality layer. (2) Anti-pattern knowledge accumulates across milestones and constrains future search, reducing wasted exploration. This is only achievable by sustained teaming and accumulated context. And (3) The context-window boundary is a hard limit—*Athena* misclassified M92's heading-level variant as safe because the prior anti-pattern was documented as "per-paragraph," a distinction that a bounded context cannot reliably track over 90+ milestones.

Finding (RQ3): *The verification phase catches substantive defects. Higher rejection rates correlate with iterative quality targets rather than project scale, and Apollo-driven feedback durably shaped Athena's planning through anti-pattern accumulation.*

4.3.4 RQ4: Cost Analysis. Across all four long-running projects, workers absorbed the dominant share of both cost and time: on average, worker agents consumed 70.6% of total token cost and 46.5% of wall-clock time. This distribution is a positive signal for continuous development: high worker concentration indicates that the system allocates the bulk of its budget to actual engineering work rather than coordination overhead. In contrast, *Athena*'s planning share in particular remained modest (9.8% of cost on average), confirming that milestone orchestration does not crowd out execution capacity as projects scale.

We define *milestone granularity* as wall-clock hours per milestone; it is set by *Athena* and varies across runs as the planner adapts decomposition to project structure and verification feedback. M2Sim milestones are the coarsest at 4.1 h each—roughly 6× coarser than RustLaTeX (0.66 h)—which explains its high per-milestone cost (\$43.9) relative to the finer-grained Projects B, C, and D (\$4.9–\$8.3 per milestone at 0.5–0.7 h each). Spearman correlation between milestone index and per-milestone cost is non-significant in all projects (Project C: $\rho = -0.06$, $p = 0.55$; Project D: $\rho = 0.06$, $p = 0.76$), indicating that cost does not systematically escalate as codebases grow.

Table 4 reports token consumption for THEBOTCOMPANY on the five benchmark problems, using output tokens (non-cached model generation) as the primary unit, matching the metric reported for

Table 4: Token consumption and wall-clock running duration for THEBOTCOMPANY on ProjDevBench problems compared with Claude Code, normalized by number of OJ submission attempts for a fair comparison. Claude Code output-token figures are taken from the ProjDevBench paper [36].

Problem #	THEBOTCOMPANY		Claude Code	
	Token/Attempt (M)	Duration	Token/Attempt (M)	Ratio
P3	1.23	19.4 hr	1.62	0.76×
P4	1.00	7.5 days	0.48	2.11×
P15	0.53	23.0 hr	0.44	1.21×
P16	0.81	9.1 days	0.15	5.41×
P17	3.50	2.1 days	1.09	3.22×
Average	1.03	0.62	1.65×	

Claude Code baselines in the ProjDevBench benchmark [36]. Because THEBOTCOMPANY and Claude Code may have used different numbers of OJ submission attempts per problem, we normalize by per-attempt output-token consumption to enable a fair comparison. Output-token consumption per attempt ranged from 0.53 M (P15) to 3.50 M (P17). On a per-attempt basis, THEBOTCOMPANY consumed fewer output tokens than Claude Code only on P3 (1.23 M vs. 1.62 M, 0.76×), while consuming more on the remaining four problems—most substantially on P16 (0.81 M vs. 0.15 M, 5.4×) and P17 (3.50 M vs. 1.09 M, 3.2×). Averaged across all problems, THEBOTCOMPANY consumed 1.03 M output tokens per attempt versus 0.62 M for Claude Code (1.65× more), reflecting the additional generation cost of coordinating multiple specialized agents within each development cycle. Per-score efficiency varied considerably: P15 achieved 22.3 K output tokens per execution-score point versus 98.1 K for P4 (4.4× worse), indicating that cycle count, rather than problem difficulty alone, is the primary driver of output-token cost.

Finding (RQ4): *Worker agents consume 70.6% of total cost across projects. Per-milestone cost is stable across the project lifetime. On a per-OJ-submission basis, THEBOTCOMPANY consumes on average 1.65× more output tokens than Claude Code (1.03 M vs. 0.62 M per attempt), reflecting the additional generation cost of multi-agent coordination within each development cycle.*

4.3.5 Output Scale. Table 5 compares the lines of code (LOC) and file count of THEBOTCOMPANY's final submissions against the average human submission scale reported in ProjDevBench [36] for the same five problems. THEBOTCOMPANY's submissions are broadly proportionate to the expected problem scale: on P3 and P4, generated LOC slightly exceeds the human average (1.48× and 1.30× respectively), while P17, the most architecturally complex problem (4,204 human-average lines across 20 files), is matched to within 7% (3,910 lines). The notable exception is P15, where THEBOTCOMPANY produces only 377 lines against a 2,094-line human average (0.18×). It is a deliberately compact, problem-specific design by THEBOTCOMPANY: after its previous submission received an OJ timeout, THEBOTCOMPANY adopted a hash-bucket implementation that exploits the P15 specification, optimizing for judged performance rather than for building a complete system (the apparent focus of human submissions); the competitive execution score confirms the solution is strong rather than incomplete. Across all five problems, THEBOTCOMPANY consistently adopts a flatter file structure than

Table 5: Final submission scale for THEBOTCOMPANY compared with average human submission scale from ProjDevBench [36]. Human figures are averages over human submissions per problem.

Prob.	THEBOTCOMPANY		Human Average		
	LOC	Files	LOC	Files	LOC Ratio
P3	705	1	475.3	1	1.48×
P4	2,161	4	1,659.0	18	1.30×
P15	377	3	2,093.9	3	0.18×
P16	1,791	7	3,139.4	11	0.57×
P17	3,910	6	4,204.3	20	0.93×

human submissions, using between one-third and one-quarter as many files on multi-file problems (P4: 4 vs. 18; P17: 6 vs. 20), reflecting its tendency to consolidate related logic rather than decompose into fine-grained modules.

4.3.6 RQ5: Failure Modes and Recovery. We derived the failure taxonomy by qualitative thematic analysis [10] of four artifact types—*Apollo* verification verdicts, issue-tracker records, CI/test output, and milestone reports—coding each distinct failure episode into one category. Two authors coded independently and reconciled disagreements by discussion; the final rubric can be found in Table 6.

Table 6 shows that failure modes are predominantly domain-specific rather than orchestration failures. The largest failure categories were accuracy/similarity regression (Projects A and C), not from deficiencies in agent coordination. Generic software engineering failures (missing requirements, architectural issues) are comparatively rare.

Recovery was effective for all in-system failures. Both *Athena* execution timeouts in Project D recovered cleanly in the next cycle: the orchestrator restarted *Athena* with full project context from the SQLite communication store, and the subsequent invocation defined a valid milestone without re-encountering the timeout. The single tooling failure in Project B (tbc-db CLI not found at Cycle 1) was self-resolved by *Athena* switching to direct `sqlite3` calls, with zero cycle waste.

One failure case represents deliberate non-recovery decisions (see §4.3.3). In Project C, a 2.12% irreducible gap in file similarity was acknowledged by *Athena*: nine distinct approaches had each been tried and documented as anti-patterns, and no unexplored strategy remained.

The cycle waste ratio (failed cycles / total) was near zero for Projects A, B, and D. In Project C, 27 fix-round milestones represent acknowledged rework cycles; if these are classified as waste, the waste ratio is $27/392 = 6.9\%$ —a modest overhead given the iterative nature of pixel-fidelity optimization. In ProjDevBench problem P4, approximately 120 late cycles targeted an unresolvable bug, yielding an estimated waste ratio of 31%.

Finding (RQ5): *Failure modes are domain-specific; orchestration-level failures (timeouts, tooling) are rare and recover within one cycle. Self-aware termination—explicit documentation of irreducible limitations—prevents runaway cost in both the long-running settings.*

4.4 Threats to Validity

Internal Validity. LLM *non-determinism* (non-zero temperature) means rerunning any project would yield different decompositions, artifacts, and quality outcomes; we report single-run results, precluding variance estimation. Researcher authorship of initial project briefs and acceptance criteria also influences development trajectory, though we mitigate this by reporting objective outcomes only (benchmark correctness, test pass rates) rather than handcrafted milestones.

External Validity. Four long-running projects skewed toward system software (simulator, database engine, compiler, interpreter) with deterministic correctness oracles may not represent the broader distribution of real-world tasks; projects with ambiguous requirements are not evaluated. RQ2 results are partially insulated from selection bias by external OJ scoring comparison with ProjDevBench. All experiments use Claude-family models; whether multi-agent coordination benefits generalize to other LLM backends remains unknown.

Construct Validity. LOC and test count are imperfect quality proxies, supplemented by domain-specific metrics (PloyBench accuracy, TPC-H query coverage, pixel similarity, OJ scores) to reduce reliance on any single measure. *Apollo*’s LLM-based milestone assessment may produce incorrect accept/reject decisions; we partially address this by anchoring acceptance criteria to executable tests over prose evaluation. Cost comparisons between THEBOTCOMPANY and single-agent baselines in the long-running track are unavailable, as multi-day Claude Code sessions under identical budgets were not feasible. A related concern is whether the projects merely reproduce existing reference implementations. Two of the four have no public reference to draw on: to our knowledge, no cycle-accurate Apple M2 simulator (Project A) and no Rust `TeX` compiler at the target fidelity (Project C) has been released when we conducted our experiments. Inspecting the orchestrator logs for all four projects, we observe agents building incrementally from an empty repository—defining data structures, implementing components, and adding tests milestone by milestone—rather than transcribing a known solution; web searches inform local decisions (e.g., API usage) rather than supplying a wholesale implementation. Where executable oracles do exist (Projects A, B, D), they constrain acceptance but not the implementation path.

5 Discussion

Sequential Phase Separation vs. Parallel Competition. Although we compared with Claude Code’s single-agent mode, the agent teams mode [4], which coordinates multiple Claude Code instances under a single team lead, represents an emerging alternative paradigm. We did not include a direct comparison because the feature remains experimental and is not publicly available through standard usage at the time we conducted our experiments, and results would be difficult to reproduce and unfair to both systems. Conceptually, however, the architectures differ in a fundamental way. Claude Code’s teams favor *parallel, competitive* coordination—agents attempt the same task and the lead selects the best result—whereas THEBOTCOMPANY enforces *sequential phase separation*: planning, implementation, and verification are isolated so that each

Table 6: Failure mode taxonomy, rubric, and counts across all experiments (RQ5). Counts are distinct failure episodes.

Failure Mode	Definition	M2Sim	GroundDB	RustLaTeX	PyInterpreter
Accuracy/similarity regression	Quantitative quality metric (timing error, pixel similarity) drops below its prior baseline	3	0	18	0
Architectural issue	Design decision requires structural rework rather than a local fix	1	0	0	0
Missing requirement	Specified capability found unimplemented at verification	0	0	5	0
Agent execution timeout	Agent invocation exceeds its wall-clock limit	0	0	0	2
Inter-milestone regression	Completed milestone breaks a previously passing one	0	0	0	1
Tooling / environment	Infrastructure failure rather than a fault in agent output	0	1	0	0
Total episodes		4	1	23	3

stage cannot be contaminated by the concerns of another. Independent verification is only meaningful when the verifier has no stake in the implementation, a principle that parallels the self-critique loops shown to improve agent quality in Reflexion [48]. We hypothesize that this separation explains the performance pattern observed in RQ2: on compact problems amenable to a single session, single execution is sufficient; on high-difficulty problems requiring iterative refinement, sequential phase separation with persistent verification feedback provides a structural advantage. As such, rigorous empirical validation of this hypothesis, ideally on a shared benchmark with both systems, is an important direction for future work. Extending THEBOTCOMPANY to run multiple workers per cycle, with the attendant merge and conflict-resolution machinery, could raise a distinct set of research questions (concurrent-edit reconciliation, verification under nondeterministic interleavings, etc.) that we leave to future work.

When does multi-agent coordination justify its cost? Our results suggest a practical crossover point. On ProjDevBench problems where the solution space is large and requires iterative exploration (P3, P17), THEBOTCOMPANY outperforms single-agent baselines despite consuming 1.65× more output tokens per attempt. On problems admitting compact, single-session solutions (P16), the multi-agent overhead reduces both score and cost-efficiency. For practitioners, this implies that multi-agent orchestration is best suited to projects where (1) the implementation cannot be completed in a single session, (2) quality criteria are continuous or iterative rather than binary, and (3) the cost of undetected regressions justifies an independent verification phase. When these conditions do not hold, a single-agent tool is likely more efficient.

Human-in-the-loop Oversight Granularity. THEBOTCOMPANY was designed for largely unattended operation, with human steering channeled through asynchronous issue filing. In practice, adding live agent output streaming to the dashboard created an implicit expectation of finer-grained control. For example, users who could watch a worker diverge naturally wanted to correct it mid-cycle. This reflects a tension in mixed-initiative systems [25]: exposing system internals raises awareness of errors but shifts the perceived locus of control, increasing felt demand for intervention. The appropriate granularity of oversight depends on user expertise and task stakes [2]—a domain expert debugging a numerical simulator has actionable knowledge that a generalist user does not. Our experience suggests a tiered model: coarse-grained issue filing for users who prefer autonomy; mid-grained pause/bootstrap for correcting runaway cycles; and potential fine-grained interruption for experts who can productively redirect an in-progress agent.

However, how to expose these tiers without overburdening the agent communication infrastructure deserves more future work. Our evaluation exercised the asynchronous channel for corrective feedback (OJ verdicts filed as issues) but deliberately held requirements fixed for reproducibility. We leave how THEBOTCOMPANY interacts and re-plans with mid-project requirement steering from new issues as future work.

6 Conclusion

We presented THEBOTCOMPANY, an open-source framework for continuous multi-agent software development built around three innovations: a repeating **Strategy** → **Execution** → **Verification** lifecycle, self-organizing worker teams that managers compose per milestone, and budget-aware scheduling for unattended operation. Evaluation across four real-world projects and five problems yields three findings. First, the verification phase adds corrective value when acceptance criteria are continuous or iterative, but acts as confirmatory overhead when deterministic oracles are available, suggesting that verification cost should be calibrated to specification clarity. Second, self-organizing teams concentrate cost on implementation work, with orchestration overhead remaining stable as projects grow. Third, in a single-run comparison, THEBOTCOMPANY achieved higher scores than the same-model single-agent baselines on high-difficulty problems requiring iterative refinement, while single-agent execution was more efficient on problems amenable to single-session solutions. Collectively, our results suggest that structural choices—phase separation, persistent state, and decoupled verification—are important complements to model capability for long-horizon software development.

Data Availability Statement

THEBOTCOMPANY is an open-source project licensed under MIT. The source code, the four long-running project specifications, and the experiment logs are archived at <https://doi.org/10.5281/zenodo.21775778> [37].

Acknowledgments

We thank our anonymous reviewers for their reviews. This work is supported by the National Science Foundation under award no. NSF-2418582 and CCF-2402804, Commonwealth Cyber Initiative under award no. HC-2Q26-037, and William & Mary’s Global Research Institute.

References

- [1] Zahra Shakeri Hossein Abad, Oliver Karras, Kurt Schneider, Ken Barker, and Mike Bauer. 2018. Task Interruption in Software Development Projects: What Makes some Interruptions More Disruptive than Others?. In *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering 2018* (Christchurch, New Zealand) (EASE '18). Association for Computing Machinery, New York, NY, USA, 122–132. doi:10.1145/3210459.3210471
- [2] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. 2019. Software Engineering for Machine Learning: A Case Study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 291–300. doi:10.1109/ICSE-SEIP.2019.00042
- [3] Anthropic. 2025. Claude Code. <https://docs.anthropic.com/en/docs/claude-code>. Accessed: March 22, 2026.
- [4] Anthropic. 2025. *Orchestrate teams of Claude Code sessions*. Anthropic. <https://code.claude.com/docs/en/agents-teams>. Accessed: March 22, 2026.
- [5] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021). doi:10.48550/arXiv.2108.07732
- [6] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D. C., Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. 2024. CodePlan: Repository-Level Coding using LLMs and Planning. *Proc. ACM Softw. Eng.* 1, FSE, Article 31 (July 2024), 24 pages. doi:10.1145/3643757
- [7] Christian Bienia, Sanjeev Kumar, Jaswinder Pal Singh, and Kai Li. 2008. The PARSEC benchmark suite: characterization and architectural implications. In *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques* (Toronto, Ontario, Canada) (PACT '08). Association for Computing Machinery, New York, NY, USA, 72–81. doi:10.1145/1454115.1454128
- [8] B. W. Boehm. 1988. A spiral model of software development and enhancement. *Computer* 21, 5 (1988), 61–72. doi:10.1109/2.59
- [9] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2188–2200. doi:10.1109/ICSE55347.2025.00157
- [10] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101. doi:10.1191/1478088706qp0630a
- [11] Edward Y Chang and Longling Geng. 2025. SagaLLM: context management, validation, and transaction guarantees for multi-agent LLM planning. *arXiv preprint arXiv:2503.11951* (2025). doi:10.48550/arXiv.2503.11951
- [12] Guangyao Chen, Siwei Dong, Yu Shu, Ge Zhang, Jaward Sesay, Börje F Karlsson, Jie Fu, and Yemin Shi. 2023. AutoAgents: A Framework for Automatic Agent Generation. *arXiv preprint arXiv:2309.17288* (2023). doi:10.48550/arXiv.2309.17288
- [13] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). doi:10.48550/arXiv.2107.03374
- [14] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. 2024. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=EHg5GDnyq1>
- [15] Markus Dreseler, Martin Boissier, Tilmann Rabl, and Matthias Uflacker. 2020. Quantifying TPC-H choke points and their optimizations. *Proc. VLDB Endow.* 13, 8 (April 2020), 1206–1220. doi:10.14778/3389133.3389138
- [16] Beck et al. 2001. Manifesto for agile software development. <https://agilemanifesto.org/>
- [17] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, 31–53. doi:10.1109/ICSE-FoSE59343.2023.00008
- [18] GitHub, Inc. 2026. GitHub Copilot: Your AI pair programmer. <https://github.com/features/copilot>. Accessed: 2026-03-23.
- [19] Michael W Godfrey and Daniel M German. 2014. On the evolution of Lehman's Laws. *Journal of Software: Evolution and Process* 26, 7 (2014), 613–619. doi:10.1002/smr.1636
- [20] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming. *arXiv preprint arXiv:2401.14196* (2024). doi:10.48550/arXiv.2401.14196
- [21] Tracy Hall, Sarah Beecham, June Verner, and David Wilson. 2008. The impact of staff turnover on software projects: the importance of understanding what makes software practitioners tick. In *Proceedings of the 2008 ACM SIGMIS CPR Conference on Computer Personnel Doctoral Consortium and Research* (Charlottesville, VA, USA) (SIGMIS CPR '08). Association for Computing Machinery, New York, NY, USA, 30–39. doi:10.1145/1355238.1355245
- [22] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 124 (May 2025), 30 pages. doi:10.1145/3712003
- [23] Israel Herraiz, Daniel Rodriguez, Gregorio Robles, and Jesus M. Gonzalez-Barahona. 2013. The evolution of the laws of software evolution: A discussion based on a systematic literature review. *ACM Comput. Surv.* 46, 2, Article 28 (Dec. 2013), 28 pages. doi:10.1145/2543581.2543595
- [24] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiaowu Zheng, Yuheng Cheng, Jinlin Hong, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VtmBAGCN7o>
- [25] Eric Horvitz. 1999. Principles of mixed-initiative user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Pittsburgh, Pennsylvania, USA) (CHI '99). Association for Computing Machinery, New York, NY, USA, 159–166. doi:10.1145/302979.303030
- [26] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 33, 8, Article 220 (Dec. 2024), 79 pages. doi:10.1145/3695988
- [27] Yoichi Ishibashi and Yoshimasa Nishimura. 2024. Self-organized agents: A llm multi-agent framework toward ultra large-scale code generation and optimization. *arXiv preprint arXiv:2404.02183* (2024). doi:10.48550/arXiv.2404.02183
- [28] Sabila Al Jannat, Ying Li, Mengyang He, Xuzhong Wang, Huizhi Zhao, Jingxiang Sun, Daoxuan Xu, Enze Xu, and Yifan Sun. 2026. Akita: A High Usability Simulation Framework for Computer Architecture. *arXiv preprint arXiv:2604.28073* (2026). doi:10.48550/arXiv.2604.28073
- [29] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQm66>
- [30] C. Larmann and V.R. Basili. 2003. Iterative and incremental developments: a brief history. *Computer* 36, 6 (2003), 47–56. doi:10.1109/MC.2003.1204375
- [31] M.M. Lehman. 1980. Programs, life cycles, and laws of software evolution. *Proc. IEEE* 68, 9 (1980), 1060–1076. doi:10.1109/PROC.1980.11805
- [32] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: May the Source Be with You! *arXiv preprint arXiv:2305.06161* (2023). doi:10.48550/arXiv.2305.06161
- [33] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustín Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097. doi:10.1126/science.abq1158
- [34] Leilei Lin, Yingming Zhou, Wenlong Chen, and Chen Qian. 2024. Think-on-process: Dynamic process generation for collaborative development of multi-agent system. *arXiv preprint arXiv:2409.06568* (2024). doi:10.48550/arXiv.2409.06568
- [35] Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. 2024. A Dynamic LLM-Powered Agent Network for Task-Oriented Agent Collaboration. In *First Conference on Language Modeling*. <https://openreview.net/forum?id=XII0Wp1XA9>
- [36] Pengrui Lu, Shiqi Zhang, Yunzhong Hou, Lyumanshan Ye, Chaoyi Huang, Zixi Chen, Ji Zeng, Hantao Jiang, Pengfei Liu, Yiwei Wang, et al. 2026. ProjDevBench: Benchmarking AI Coding Agents on End-to-End Project Development. *arXiv preprint arXiv:2602.01655* (2026). doi:10.48550/arXiv.2602.01655
- [37] Wenhan Lyu, Yiran Xiao, Yating Zhang, and Yu Sun. 2026. TheBotCompany Source Code and Experiment Logs. doi:10.5281/zenodo.21775778
- [38] Robert M. Metcalfe and David R. Boggs. 1976. Ethernet: distributed packet switching for local computer networks. *Commun. ACM* 19, 7 (July 1976), 395–404. doi:10.1145/360248.360253
- [39] Minh Huynh Nguyen, Thang Phan Chau, Phong X. Nguyen, and Nghi D. Q. Bui. 2025. AgileCoder: Dynamic Collaborative Agents for Software Development based on Agile Methodology. In *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*, 156–167. doi:10.1109/Forge66646.2025.00026
- [40] N. Nurmilani, D. Zowghi, and S. Powell. 2004. Analysis of requirements volatility during software development life cycle. In *2004 Australian Software Engineering Conference. Proceedings*, 28–37. doi:10.1109/ASWEC.2004.1290455
- [41] OpenAI. 2025. Introducing Codex. <https://openai.com/index/introducing-codex/>. Accessed: March 22, 2026.
- [42] Melissa Z Pan, Mert Cemri, Lakshya A Agrawal, Shuyi Yang, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Kannan Ramchandran,

- Dan Klein, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. 2025. Why Do Multiagent Systems Fail?. In *ICLR 2025 Workshop on Building Trust in Language Models and Applications*. <https://openreview.net/forum?id=wM521FqPvI>
- [43] Louis-Noël Pouchet. 2012. PolyBench/C: The Polyhedral Benchmark Suite. <https://www.cs.colostate.edu/~pouchet/software/polybench/>. Accessed: 2026-03-23.
- [44] Chen Qian, Yufan Dang, Jiahao Li, Wei Liu, Zihao Xie, YiFei Wang, Weize Chen, Cheng Yang, Xin Cong, Xiaoyin Che, Zhiyuan Liu, and Maosong Sun. 2024. Experiential Co-Learning of Software-Developing Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 5628–5640. doi:10.18653/v1/2024.acl-long.305
- [45] Chen Qian, Jiahao Li, Yufan Dang, Wei Liu, YiFei Wang, Zihao Xie, Weize Chen, Cheng Yang, Yingli Zhang, Zhiyuan Liu, and Maosong Sun. 2024. Iterative Experience Refinement of Software-Developing Agents. *arXiv preprint arXiv:2405.04219* (2024). doi:10.48550/arXiv.2405.04219
- [46] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 15174–15186. doi:10.18653/v1/2024.acl-long.810
- [47] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2024. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2024). doi:10.48550/arXiv.2308.12950
- [48] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 8634–8652. https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf
- [49] Vali Tawosi, Keshav Ramani, Salwa Alami, and Xiaomo Liu. 2025. ALMAS: an Autonomous LLM-based Multi-Agent Software Engineering Framework. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*. 287–290. doi:10.1109/ASEW67777.2025.00059
- [50] Hàn Thê Thành. 2004. The pdfTeX user manual. *TUGboat* 25, 1 (2004), 121–127. <https://www.tug.org/applications/pdftex/>. Accessed: 2026-03-24.
- [51] Qian Wang, Tianyu Wang, Zhenheng Tang, Qinbin Li, Nuo Chen, Jingsheng Liang, and Bingsheng He. 2025. MegaAgent: A Large-Scale Autonomous LLM-based Multi-Agent System Without Predefined SOPs. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 4998–5036. doi:10.18653/v1/2025.findings-acl.259
- [52] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2024. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *arXiv preprint arXiv:2407.16741* (2024). doi:10.48550/arXiv.2407.16741
- [53] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 50528–50652. doi:10.52202/079017-1601
- [54] Zheng Yu, Ziyi Guo, Yuhang Wu, Jiahao Yu, Meng Xu, Dongliang Mu, Yan Chen, and Xinyu Xing. 2025. PATCHAGENT: a practical program repair agent mimicking human expertise. In *Proceedings of the 34th USENIX Conference on Security Symposium* (Seattle, WA, USA) (SEC '25). USENIX Association, USA, Article 226, 20 pages.
- [55] Mario Zechner. 2026. pi-ai: Unified LLM API with automatic model discovery and tool calling. <https://github.com/badlogic/pi-mono/tree/main/packages/ai>. Accessed: 2026-03-23.
- [56] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 2471–2484. doi:10.18653/v1/2023.emnlp-main.151
- [57] D. Zowghi and N. Nurmiliani. 2002. A study of the impact of requirements volatility on software project performance. In *Ninth Asia-Pacific Software Engineering Conference, 2002*. 3–11. doi:10.1109/APSEC.2002.1182970

Received 2026-03-26; accepted 2026-06-18